**Proceedings of the ASME 2026 International Design Engineering Technical Conferences and Computers and Information in Engineering Conference**
**IDETC-CIE2026**
**August 23–26, 2026, Houston, TX, USA**

**IDETC-CIE2026-XXXXX**

# FROM DESCRIPTIONS TO SOURCEABLE PARTS: CATALOG-GROUNDED RETRIEVAL FOR MECHANICAL DESIGN

**Chancel Lavoie[1], Levent Burak Kara[1]**

[1]Carnegie Mellon University, Pittsburgh, PA

## ABSTRACT

*AI engineering agents are beginning to generate mechanical part and component descriptions from textual requirements, but these outputs are difficult to build when they are not grounded in existing, sourceable, trusted catalog parts. Product-data APIs can provide live links, prices, datasheets, and CAD files once part numbers are known, but they do not solve the upstream problem of converting a component description into the correct part-number set. We present an open-source Model Context Protocol server for headless navigation of McMaster-Carr that converts sufficiently detailed component descriptions into matching catalog part numbers. The system does not rely on a hand-coded ontology of catalog attributes. Instead, it renders live catalog pages, extracts visible product-table schemas, uses a language model to map requested constraints onto live field names and values, and then performs deterministic row filtering. The output is simply the set of catalog parts that satisfy the grounded description. In final benchmark runs, the system recovered 250/250 exact part numbers across 25 catalog categories, returned the expected multi-part sets in 50/50 under-specified cases, and returned no part in 25/25 nonexistent cases. By making part sourcing callable by agents, the method enables downstream workflows in which assemblies are crafted from off-the-shelf parts and automatically linked to product data, prices, datasheets, CAD, and bills of materials.*

**Keywords:** AI engineering agents, design automation, part sourcing, model context protocol, catalog grounding, bill of materials, CAD retrieval

## 1. INTRODUCTION

AI engineering agents are increasingly being used to generate mechanical part requirements, component selections, fixtures, subsystems, and assembly concepts from text. For these systems to become useful in real engineering workflows, their outputs must connect to existing parts that can actually be purchased, trusted, inspected, and inserted into CAD. A generated component description is useful to a human engineer, but procurement, bill-of-materials generation, and CAD retrieval still require exact catalog identifiers. This mismatch creates a practical bottleneck between language-level engineering intent and sourceable mechanical parts. Solving part sourcing is therefore a prerequisite for agents that craft larger assemblies from off-the-shelf components.

The status quo separates broad catalog search from product-data retrieval. McMaster-Carr is a widely used mechanical component catalog, and its Product Information API retrieves product information, live links, prices, images, CAD, and datasheets once a subscribed part number is known [1]. Existing McMaster-Carr agent tooling also supports search and part-number lookup, but it does not provide a general exact-part resolver that navigates rendered catalog pages, interprets live product-table schemas, and returns the full set of parts satisfying a textual requirement. As a result, the most valuable downstream outputs remain gated on a missing upstream step: discovering the part number.

The gap is challenging because industrial catalogs are not organized around a single fixed schema. A screw might be filtered by material, thread size, length, drive style, package quantity, and finish. A switch might be filtered by circuit, terminal count, maintained or momentary action, panel thickness, voltage, and terminal style. A conveyor roller, spring, hinge, tape, bearing, or filter has a different set of discriminating fields. A general system cannot be built by hard-coding every possible catalog attribute, and a ranked answer is not sufficient for exact procurement. The correct output is the complete set of catalog rows that satisfy the description.

In this work, we present a dynamic catalog-grounding method exposed through a Model Context Protocol (MCP) server. The input is a textual mechanical component description, optionally with a broad search hint, and the output is the set of part numbers whose live catalog rows satisfy the grounded constraints. The system renders the current catalog page, extracts product rows and visible column names, summarizes the live schema, uses a language model only to align requested constraints with those live fields and values, and then filters rows programmatically. The language model is not asked to hallucinate or rank part numbers; it is asked to produce grounded matchers against fields actually present on the page. Once part numbers are recovered, existing product-data APIs can automatically attach live links, price information, datasheets, and CAD files to generated component or assembly descriptions.

Our main contributions are:

1. a reusable MCP server for headless McMaster-Carr navigation, rendered schema extraction, and exact-part resolution;
2. a dynamic schema-grounding method that maps textual requirements to live catalog fields and filters rows without a hand-coded catalog ontology; and
3. a part-number retrieval contract that returns all catalog parts satisfying the requirement for downstream product-data, CAD, datasheet, and BOM retrieval.

## 2. RELATED WORK

Our review focuses on three bodies of related work: tool protocols for connecting language models to external systems, McMaster-Carr product-data interfaces, and prior McMaster-Carr agent tooling.

### 2.1. Model Context Protocol for Engineering Tools

The Model Context Protocol standardizes how language-model applications connect to external context and tools [2]. MCP servers can expose resources, prompts, and tools; tools are the primitive used when a model needs to execute an action such as querying a database, calling an API, or driving an external system [3]. Our implementation uses MCP as the integration layer so that the same part-sourcing capability can be called by any compatible agent host.

MCP is useful here because part sourcing is not simply text generation. It requires a persistent browser session, rendered-page inspection, navigation, timeouts, and a structured result contract. These actions are better exposed as tools than hidden inside a prompt.

### 2.2. McMaster-Carr Product Data

McMaster-Carr's Product Information API is available to approved customers and includes requests for product information, price, image retrieval, CAD retrieval, and datasheet retrieval once a subscribed part number is known [1]. This makes part-number discovery the critical upstream step. Our system is designed to fill that step and then hand part numbers to an approved product-data pipeline.

### 2.3. Existing McMaster-Carr Agent Tooling

A prior public repository, `mjbraun/mcmaster-agent`, exposes an MCP server for searching McMaster-Carr and looking up product information from part numbers [4]. Its documentation describes product lookup, search, URL generation, SeleniumBase UC mode, cookie persistence, and a limitation that a visible browser window may briefly appear. That tool is useful for basic search and product lookup. The work reported here differs in three ways: it is headless-only, it exposes stateful navigation and schema extraction, and its resolver returns the set of part numbers that satisfy dynamically filtered live catalog schemas.

## 3. SYSTEM DESIGN

### 3.1. MCP Server Surface

The package exposes a Python MCP server over stdio. The main user-facing tool is `mcmaster_find_exact_part`; the remaining tools expose the browser and extraction state needed for inspection, recovery, and agent-driven navigation when a first search page is insufficient. The public tools are:

- `mcmaster_find_exact_part`: resolve a detailed text description to the matching set of catalog part numbers;
- `mcmaster_extract_schema`: open or search a page and return dynamically extracted filters, table columns, row attributes, and part-number rows;
- `mcmaster_search`, `mcmaster_open`, `mcmaster_follow_link`, `mcmaster_current_page`, and `mcmaster_back`: provide stateful headless navigation through rendered pages;
- `mcmaster_find_parts`: broad search returning part numbers found during rendered search/category navigation; and
- `mcmaster_url`: generate product or search URLs without launching a browser.

These support tools are not separate research claims. They make the resolver usable by external agents: an agent can inspect the evidence behind a result, move through category pages, extract the current schema, and retry with better context without reimplementing browser automation. The implementation is packaged as `mcmaster-navigator-mcp`, requires Python 3.11 or newer, and uses SeleniumBase UC mode with headless Chrome. Model credentials are supplied at runtime rather than embedded in the package.

### 3.2. Headless Browser and Rendered-Page Extraction

McMaster-Carr pages are rendered web pages with product tables, category links, filters, and product detail pages. The navigator opens search URLs, optionally auto-drills through category links, and snapshots the rendered HTML. The extractor returns part numbers, product rows, row attributes, evidence text, navigable links, and schema objects containing filters, table columns, and part-number rows.

The browser is isolated by default with a temporary Chrome profile. The server serializes browser work through a single worker so browser state remains coherent across navigation calls. Browser logs are kept off MCP protocol stdout.

### 3.3. Dynamic Exact-Part Resolution

Figure 1 summarizes the exact-part resolver. The resolver normalizes the description into broad search roots and literal constraints, searches the generated roots headlessly, extracts rows from product hits and schema tables, summarizes the live schema, asks the language model to map requested constraints to live fields and accepted values, and applies the resulting matchers with Python row filtering. If filtering conflicts with the live schema, the language model repairs the mapping from the extracted field/value summary. If one row remains, the resolver runs a final strict

**Text requirement** → search roots and constraints → headless rendered pages → live rows and schema fields → LLM field/value matchers → deterministic row filtering → matching part-number set.

**FIGURE 1: Dynamic exact-part resolution. The language model maps constraints to live page fields; Python filtering determines the returned set of matching part numbers.**

verification check against that row. If required constraints are missing or contradicted, the row is removed from the result set.

This design lets the system generalize across product families because the catalog schema is discovered at runtime. There is no static list of all possible attributes for screws, springs, switches, hinges, bearings, hose fittings, raw materials, or other categories.

### 3.4. Assembly-Level Use Case

The core problem is part sourcing. Assembly workflows become possible by treating each off-the-shelf line item as an independent exact-part resolution problem. A component requirement is sent to `mcmaster_find_exact_part`, and the returned matching set is carried forward. Downstream systems can retrieve product data, live links, price, image, CAD, and datasheet information for returned part numbers, preserve alternatives for later selection, or revise line items whose descriptions do not match any catalog row.

This line-item independence allows parallel sourcing agents. Compatibility checking can be layered above the resolver, but the core tool intentionally solves the narrower and measurable problem of finding the catalog part numbers that match stated requirements.

## 4. EXPERIMENT DESIGN

### 4.1. Exact Part Recovery

The exact recovery benchmark begins with known part numbers sampled from rendered McMaster-Carr search/category pages. Seed queries cover broad catalog areas including door hinges, toggle switches, welding clamps, socket head cap screws, air filter cartridges, drawer slides, compression springs, cartridge heaters, grease fittings, digital calipers, PVC tubing, timing belt pulleys, eye bolts, aluminum bar, wire rope clips, bearings, guide rails, pneumatic cylinders, double-sided tape, beakers, motors, and conveyor rollers.

For each sampled part number, the benchmark builds a textual description from the rendered product page or product-table row. The part number is withheld from the resolver. A case is counted correct if the returned set contains exactly one part number and that part number is the target. The final exact-recovery run used 250 cases, 25 catalog categories, `gpt-5.4-mini`, page budget 8, auto-drill depth 2, maximum schema rows 700, and two LLM search roots.

### 4.2. Match-Set Size Checks

The near multi-match benchmark starts from successful exact cases, collects live schema rows around the target, then removes one discriminator from the description so that two to four parts are expected to remain. A case passes when the resolver returns the expected matching set, includes the original target part, and does not collapse the result to a single part.

The nonexistent benchmark starts from exact descriptions and appends impossible required constraints such as an unobtainable material, impossible color, and impossible size. The broad multi-match benchmark gives underspecified category queries such as “double sided tape,” “ball bearing,” “door hinges,” or “toggle switch.” These cases test whether the system returns the matching set instead of forcing a single best guess.

## 5. EXPERIMENTAL RESULTS

Table 1 reports the final benchmark runs. The exact-recovery benchmark returned exactly one part number for all 250 cases, and that part number was the target in every case. Median exact-case latency was 54.448 s, p90 latency was 85.960 s, p95 latency was 161.763 s, and maximum latency was 466.125 s. Token use averaged 7,449.96 tokens per case, with median 6,962.5, p90 10,563.9, p95 11,183.6, and maximum 18,300.

The near multi-match benchmark returned candidate counts from 2 to 4, with mean 2.24. The expected matching set coverage was 1.0 in all cases. The nonexistent benchmark returned empty part-number sets for all 25 cases. The broad multi-match benchmark returned multiple part numbers for all 25 cases. Returned candidate counts ranged from 2 to 456, with median 86 and mean 128.88.

The absence of misses across these categories is evidence that runtime schema extraction can handle substantially different product families. It is not proof of complete coverage of every McMaster-Carr page or every possible human description. The strongest defensible claim is that dynamic schema extraction and deterministic filtering can recover exact part numbers across diverse catalog families when descriptions contain sufficient identifying information.

## 6. CONCLUSION

This work turns a practical part-sourcing bottleneck into a measurable tool interface. A design agent can describe a required component, but a downstream product information API needs a part number. The proposed MCP server bridges that gap by headlessly navigating McMaster-Carr, extracting live product schemas, grounding textual constraints to live fields, and filtering rows deterministically. In final benchmark runs, the system recovered 250/250 exact part numbers across 25 catalog categories, returned expected multi-part sets when requirements were under-specified, and returned no part when requirements could not be satisfied. Reliable part sourcing can let agents compose off-the-shelf components into larger designs while automatically attaching BOM entries, CAD, datasheets, prices, live links, and product data.

## 7. DISCUSSION

### 7.1. Why Dynamic Schemas Matter

The benchmark covers product families whose discriminating attributes differ substantially. A static parser with a fixed list of fields would need to know domain-specific names and value formats for hinges, switches, bearings, adhesives, filters, raw materials, pneumatic cylinders, motors, and conveyor rollers. The

**TABLE 1: Final benchmark results. Tokens are total recorded model tokens for each final run.**

| Evaluation | Cases | Expected count | Pass criterion | Passed | Pass rate | Tokens | Mean s/case |
|---|---|---|---|---|---|---|---|
| Exact single-part recovery | 250 | 1 | Target returned as only part | 250/250 | 100% | 1,862,490 | 71.038 |
| Near multi-match | 25 | 2–4 | Expected matching set returned | 25/25 | 100% | 187,701 | 64.325 |
| Nonexistent requirements | 25 | 0 | No part returned | 25/25 | 100% | 268,293 | 75.348 |
| Broad multi-match | 25 | > 1 | Multiple parts returned | 25/25 | 100% | 90,950 | 50.654 |
| Total final reported set | 325 | mixed | Task-specific | 325/325 | 100% | 2,409,434 | – |

**TABLE 2: Exact-recovery category coverage. Each category had 100% single-part recovery in the final run.**

| Category | Cases | Single |
|---|---|---|
| Adhesives | 8 | 8 |
| Bearings | 8 | 8 |
| Building & Grounds | 20 | 20 |
| Conveying | 8 | 8 |
| Electrical & Lighting | 6 | 6 |
| Fabricating | 19 | 19 |
| Fastening & Joining | 20 | 20 |
| Filtering | 21 | 21 |
| Furniture & Storage | 14 | 14 |
| Hand Tools | 8 | 8 |
| Hardware | 8 | 8 |
| Heating & Cooling | 8 | 8 |
| Lab Supplies | 8 | 8 |
| Linear Motion | 7 | 7 |
| Lubricating | 8 | 8 |
| Measuring & Inspecting | 8 | 8 |
| Motors | 8 | 8 |
| Pipe/Tubing/Hose/Fittings | 8 | 8 |
| Plumbing & Janitorial | 8 | 8 |
| Pneumatics | 8 | 8 |
| Power Transmission | 8 | 8 |
| Pulling & Lifting | 8 | 8 |
| Raw Materials | 8 | 8 |
| Shipping | 8 | 8 |
| Suspending | 7 | 7 |

dynamic approach avoids that requirement. The system only needs a general row representation and a way to align user constraints with fields currently visible on the page.

### 7.2. Why Returning the Set Matters

Procurement workflows need faithful part sets. Returning a single best guess for "ball bearing" is worse than returning the matching catalog rows, because the description does not specify shaft diameter, housing diameter, width, seal type, load capacity, or material. The multi-match and zero-match experiments show that the resolver can return all matching parts, or none, without false precision.

### 7.3. Integration with Product-Data APIs

Once part numbers are known, product-data retrieval is a conventional API task. The McMaster-Carr Product Information API includes endpoints for product information, price, images, CAD, and datasheets [1]. This separation lets the proposed MCP server remain focused on the hard upstream retrieval problem: finding the part numbers from text and navigation.

## 8. LIMITATIONS AND FUTURE WORK

The benchmark descriptions were generated from rendered catalog rows and product pages. They are specific and often include labels such as family, group, selected option, and attribute names. This is a valid test of schema grounding and exact retrieval, but it is not the same as testing noisy human design notes.

The system depends on rendered website structure. If McMaster-Carr changes page layout, table markup, anti-automation behavior, or part-number presentation, extraction may require maintenance.

The system is not affiliated with McMaster-Carr and should be used in accordance with applicable terms and approved API access. The official product information API requires approved customer access and subscription to product information.

Latency is dominated by headless browsing. The mean exact-recovery latency in the final run was 71.038 seconds per case. Parallel line-item sourcing can reduce assembly-level wall-clock time, but single-line resolution remains slow relative to a pure API call.

Future work should evaluate noisier human-written requirements, demonstrate full assembly-level sourcing with compatibility checks across line items, and measure robustness over time as catalog pages change.

## REFERENCES


[1] McMaster-Carr. "McMaster-Carr Product Information API." McMaster-Carr (2026). Accessed 2026-05-11, URL https://www.mcmaster.com/help/api/.

[2] Model Context Protocol. "Model Context Protocol Specification." Model Context Protocol (2024). Accessed 2026-05-11, URL https://modelcontextprotocol.io/specification/2024-11-05/index.

[3] Model Context Protocol. "Tools." Model Context Protocol (2026). Accessed 2026-05-11, URL https://modelcontextprotocol.io/specification/draft/server/tools.

[4] Braun, Michael J. "McMaster-Carr MCP Server." GitHub (2026). Accessed 2026-05-11, URL https://github.com/mjbraun/mcmaster-agent.